VLA Final Project 2026

VLA Final Project 2026

Vision-Language-Action for Autonomous Driving

---

1. Project Introduction

Vision-Language-Action (VLA) models represent a cutting-edge approach in robotics and autonomous driving, combining visual perception, natural language understanding, and action generation into a unified framework. Unlike traditional autonomous driving systems that rely on separate perception, planning, and control modules, VLA models can directly map visual observations and language instructions to executable actions.

Key Advantages of VLA: - **Unified Architecture**: Single model for perception, reasoning, and action - **Natural Language Interface**: Control robots with text commands - **Strong Generalization**: Pre-trained on large-scale robot data - **Open-Source Available**: Multiple open-source implementations

Main Open-Source VLA Repositories:

Model	GitHub	Paper	Features
OpenVLA	openvla/openvla ⭐	arXiv:2406.09246	7B params, pretrained on 970k robot episodes
RT-1	google-research/robotics_transformer	arXiv:2212.06817	Robotics Transformer for real tasks
LLaVA	haotian-liu/LLaVA ⭐	arXiv:2304.08485	Large language and vision assistant
VIMA	vimalabs/VIMA	arXiv:2210.11711	Generalist robot agent with multimodal prompts
RT-2	kyegomez/RT-2 *(third-party)*	arXiv:2307.15868	Vision-language-action at scale

Model	GitHub	Paper	Features
PaLM-E	kyegomez/PALM-E *(third-party)*	arXiv:2303.03378	Embodied multimodal language model

CARLA Simulator: - Website: https://carla.org/ **- Documentation**: https://carla.readthedocs.io/ **- GitHub**: carla-simulator/carla

---

2. Project Options

Choose ONE of the following three tasks:

- **Task 1**: VLA for Basic Driving Commands (Basic)
- **Task 2**: VLA for Complex Navigation (Intermediate)
- **Task 3**: VLA for Multi-Task Driving (Advanced)

---

3. Task 1: VLA for Basic Driving Commands (Basic Level)

3.1 Introduction

In this task, you will implement a VLA-based system that can execute basic driving commands in CARLA simulator. The system should understand natural language commands and execute corresponding actions.

3.2 Project Description

Implement a VLA system for basic driving commands: **- Commands**: “turn left”, “turn right”, “go straight”, “stop”, “slow down”, “speed up” **- Environment**: CARLA simulator with basic town map **- Goal**: Achieve >80% success rate on command execution

3.3 Requirements

1. Set up CARLA simulator (refer to CARLA documentation)
2. Set up OpenVLA (refer to OpenVLA GitHub)
3. Implement VLA inference pipeline for driving commands
4. Test with at least 5 different commands
5. Record success rate and analyze failures

3.4 Recommended Approach

Step 1: Install CARLA and OpenVLA following official documentation

Step 2: Create command-to-action mapping

```python
# Example: Use VLA to predict actions from images + language commands
from openvla import OpenVLA
model = OpenVLA.from_pretrained("openvla/openvla-7b")

# Input: image + text command
# Output: steering, throttle, brake values
```

Step 3: Test in CARLA with various commands

Step 4: Analyze success rate and failure cases

3.5 Evaluation Criteria

Criteria	Points
Environment setup	15
VLA inference implementation	35
Command execution success rate	25
Report quality	15
Code documentation	10

---

4. Task 2: VLA for Complex Navigation (Intermediate Level)

4.1 Introduction

In this task, you will go beyond basic commands and implement a VLA system for complex navigation instructions. The system should handle multi-step commands like “navigate to the traffic light and turn right” or “park between the two vehicles”.

4.2 Project Description

Implement a VLA system for complex navigation: - **Commands**: Multi-step navigation, conditional actions, spatial references - **Environment**: CARLA with varied scenarios - **Goal**: Achieve >70% success rate on complex navigation tasks

4.3 Requirements

1. Complete Task 1 or demonstrate equivalent capability
2. Design data collection pipeline in CARLA
3. Collect training data for complex navigation commands
4. Fine-tune or adapt VLA for navigation tasks
5. Evaluate in varied scenarios

4.4 Recommended Approach

Step 1: Design navigation command taxonomy - Multi-step commands: “turn left, then go straight” - Conditional commands: “if traffic light is red, stop” - Spatial references: “park between the cars”

Step 2: Collect data in CARLA - Use CARLA’s Python API to collect image-action pairs - Annotate with natural language commands

Step 3: Fine-tune VLA

```
# Example: Fine-tune OpenVLA on your navigation data
from openvla import OpenVLA
from peft import LoraConfig, get_peft_model

model = OpenVLA.from_pretrained("openvla/openvla-7b")
# Apply LoRA fine-tuning
```

Step 4: Evaluate and compare with baselines

4.5 Other VLA Models to Consider

Model	GitHub	适合任务
LLaVA	haotian-liu/LLaVA	Visual reasoning
VIMA	vimalabs/VIMA	Multimodal prompting
RT-1	robotics-transformer/RT-1	Real-world robotics

4.6 Evaluation Criteria

Criteria	Points
Data collection pipeline	20
Model fine-tuning/adaptation	30
Experimental evaluation	25
Report quality	15
Code quality	10

---

5. Task 3: VLA for Multi-Task Driving (Advanced Level)

5.1 Introduction

In this advanced task, you will design a complete VLA system capable of handling multiple driving tasks through natural language commands. This includes navigation, obstacle avoidance, emergency braking, parking, and lane changing.

5.2 Project Description

Design a multi-task VLA driving system: - **Tasks**: Navigation, obstacle avoidance, parking, lane changing, emergency response - **Commands**: Complex, multi-step, context-dependent - **Goal**: Achieve >65% success rate across all task categories

5.3 Requirements

1. Design comprehensive task taxonomy
2. Create multi-task dataset in CARLA
3. Implement multi-task learning framework
4. Design evaluation metrics for each task
5. Conduct ablation studies
6. Compare with task-specific baselines

5.4 Recommended Approach

Step 1: Design task taxonomy

```
- Navigation: "go to the parking lot"
- Obstacle avoidance: "avoid the pedestrian"
- Parking: "park between the two cars"
- Lane changing: "change to the left lane"
- Emergency: "brake immediately"
```

Step 2: Create multi-task dataset - Collect diverse scenarios in CARLA - Annotate with task labels and commands - Balance dataset across tasks

Step 3: Multi-task learning

```
# Example: Multi-task VLA training
model = OpenVLA.from_pretrained("openvla/openvla-7b")
# Use task-specific heads or prompts
```

Step 4: Comprehensive evaluation - Per-task metrics - Cross-task generalization - Failure analysis

5.5 Advanced VLA Resources

Resource	Link	Description
Open X-Embodiment	robotics-transformer-x.github.io	Largest robot dataset
RT-2 Paper	arXiv:2307.15868	VLA at scale
PaLM-E Paper	arXiv:2303.03378	Embodied multimodal LM

5.6 Evaluation Criteria

Criteria	Points
Task design and taxonomy	15
Dataset creation	20
Model implementation	25
Experimental evaluation	20
Report quality	15
Innovation	5

---

6. Project Steps

Step 1: Environment Setup (Week 1)

CARLA Installation: - Follow official guide: https://carla.readthedocs.io/en/latest/start_quickstart/

OpenVLA Setup: - Follow GitHub: https://github.com/openvla/openvla

Verify Installation:

```
# Test CARLA
python -c "import carla; print('CARLA OK')"

# Test OpenVLA
python -c "from openvla import OpenVLA; print('OpenVLA OK')"
```

Step 2: Data Collection (Week 2)

Use CARLA Python API to collect data:

```
import carla

client = carla.Client('localhost', 2000)
world = client.get_world()

# Spawn vehicle
vehicle_bp = world.get_blueprint_library().filter('model3')[0]
spawn_point = world.get_map().get_spawn_points()[0]
vehicle = world.spawn_actor(vehicle_bp, spawn_point)

# Attach camera
camera_bp = world.get_blueprint_library().find('sensor.camera.rgb')
camera = world.spawn_actor(camera_bp, carla.Transform(),
        attach_to=vehicle)
```

Step 3: Model Implementation (Week 3-4)

Choose and implement VLA model: - OpenVLA (recommended for beginners) - RT-1/RT-2 (more advanced) - LLaVA-based approach (alternative)

Step 4: Evaluation (Week 5)

Evaluate using metrics: - Success rate - Path efficiency - Collision rate - Command understanding accuracy

Step 5: Report Writing (Week 6)

Report structure: 1. Introduction 2. Related Work 3. Methodology 4. Experiments 5. Results 6. Conclusion 7. References

---

7. Grading

Component	Task 1	Task 2	Task 3
Environment Setup	15	10	5
Implementation	35	30	25
Experiments	25	30	35
Report	15	20	20
Documentation	10	10	10
Innovation Bonus	-	-	5

---

8. Resources

VLA Models

1. **OpenVLA**: https://github.com/openvla/openvla ⭐ *(推荐入门)*
2. **RT-1**: https://github.com/google-research/robotics_transformer
3. **LLaVA**: https://github.com/haotian-liu/LLaVA
4. **VIMA**: https://github.com/vimalabs/VIMA
5. **RT-2** (third-party): https://github.com/kyegomez/RT-2
6. **PaLM-E** (third-party): https://github.com/kyegomez/PALM-E

CARLA Simulator

- **Website**: https://carla.org/
- **Documentation**: https://carla.readthedocs.io/
- **GitHub**: https://github.com/carla-simulator/carla

Datasets

- **Open X-Embodiment**: https://robotics-transformer-x.github.io/
- **CARLA Dataset**: Built-in scenarios

Key Papers

1. OpenVLA: An Open-Source Vision-Language-Action Model (2024)
2. RT-2: Vision-Language-Action Models (2023)
3. RT-1: Robotics Transformer (2022)
4. PaLM-E: An Embodied Multimodal Language Model (2023)
5. VIMA: Generalist Robot Agent (2022)

---

9. FAQ

Q: Do I need a GPU? A: Yes, GPU is required for VLA inference. 16GB VRAM recommended.

Q: Can I use other VLA models? A: Yes, you can use any open-source VLA model.

Q: What if CARLA is too slow? A: Use lower graphics settings or cloud services.

Q: Is prior robotics experience required? A: No, but basic deep learning knowledge is helpful.

Q: Can I work in groups? A: Please confirm with instructor.

---

10. Important Dates

- **Project Start**: [Date]
- **Midterm Check**: [Date]
- **Final Submission**: [Date]

Contact

- **Instructor**: [Your Name]
- **Email**: [Your Email]
- **Office Hours**: [Time and Location]

Good luck with your project! 🚗🤖